



SYSTEM DESIGN ASSISTANT · DOCUMENT 8

What the engine **covers** — and why

This document explains, in plain terms, which system-design questions the tool can answer, the precise reason each one is or is not in reach, and where the boundary of the tool deliberately lies. It is written to be read on its own — every term it relies on is defined in the glossary at the end.

• About this document

WHAT IS THE SYSTEM DESIGN ASSISTANT?

The System Design Assistant (SDA) is a tool for designing software architecture on a canvas and then **checking the design by computing it**, rather than merely drawing it. You place building blocks (a load balancer, a database, a queue) and connect them; the tool calculates how the resulting system would behave — how much traffic it can carry, how slow it would be, how reliable, how expensive — and tells you, with reasons, where it would fail to meet your targets. Run in reverse, it can even propose or repair designs.

Who this is for. Anyone deciding what the tool should and should not do: an architect evaluating whether SDA can answer their question, or a contributor deciding whether a new feature belongs. You do not need to know the tool's internals to read it.

How to read it. Sections build on each other: first the class of mistakes SDA exists to prevent, then how it is built, then the single rule that decides what is coverable, then the coverage itself, a worked example, the boundary of scope, and finally a glossary. Abbreviations are spelled out the first time they appear and collected in the glossary.

1 The problem SDA exists to solve

Most system-design mistakes are not drawing mistakes. They are mistakes of arithmetic that never happened — assumptions that were written down but never checked.

A diagram can look perfectly reasonable and still describe a system that cannot work. The classic failures all share one cause: a number that was never actually computed across the whole design. For example:

- **Reasoning about averages instead of the slow cases.** A team estimates "average response time is about 60 milliseconds" by adding up average times, and ships. In production the slow end of the distribution — the 1-in-100 request — is several times worse, because requests queue. Users feel the slow tail, not the average.
- **Assuming a chain runs as fast as its quickest part.** A request passes through five components. Throughput is limited by the *slowest* one, not the fastest; adding capacity anywhere else changes nothing. Teams routinely spend effort on the wrong tier.
- **Forgetting that reliability multiplies downward.** Five components that are each "99.9% available" do not give a 99.9% system; in a chain their availabilities multiply to roughly 99.5%, and a single component with no backup drags the whole thing down.
- **Overlooking hard limits.** A PostgreSQL database accepts only 100 simultaneous connections by default. Under load the connection pool is exhausted and the application stalls — a limit that was knowable on day one.

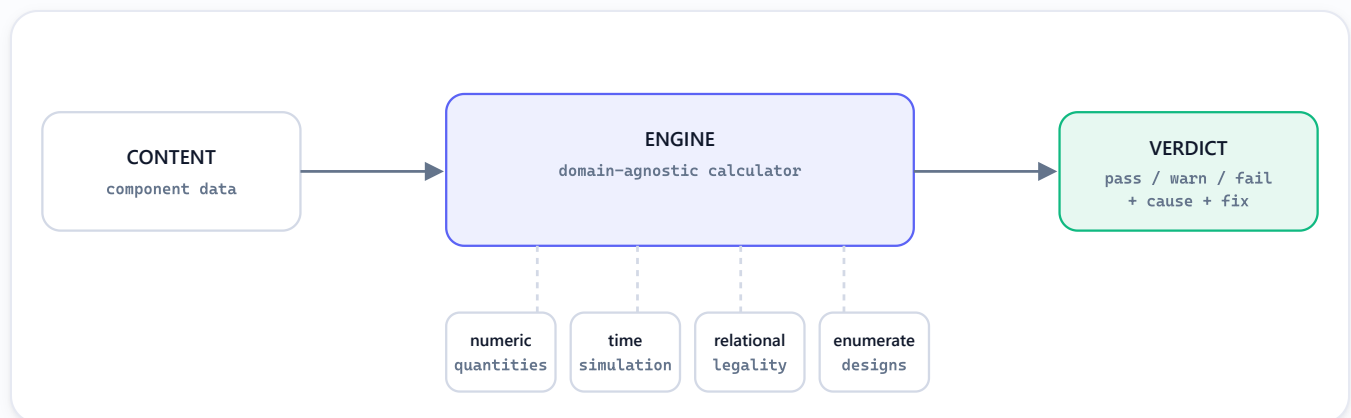
SDA attacks exactly this class of error. It does not try to judge whether your business logic is correct or your code is good. It answers the **quantitative envelope** — capacity, speed, reliability, cost, and legality of the wiring — by actually computing it across the whole design, and it does so **honestly**: it distinguishes a value you assumed from a value it derived, and when it cannot know something it says *unknown* rather than guessing.

2 How SDA is built — one engine, four lenses

A deliberate split keeps the tool honest and general: a calculation engine that knows no cloud vocabulary, fed by content that supplies all the meaning.

The **engine** is domain-agnostic: it computes over an abstract graph of components and their properties and knows nothing about "load balancers" or "databases" — those words never appear in it. All cloud meaning lives in **content**: data files that describe each component (its limits, its formulas, the protocols it speaks). The same engine therefore works for any kind of system; teaching it new components means adding data, never changing the engine.

The engine answers questions through four independent solvers — think of them as four lenses on the same design. A numeric solver computes quantities; a time-domain simulator computes behaviour over time; a relational solver checks legality; and an enumerator generates candidate structures. Their output is an honest **verdict** for every target you set.



Content (the data) feeds the engine; the engine runs four solvers over one shared model and returns an explanatory verdict.

3 The rule that decides what is coverable

Coverage is not ad hoc. A question is answerable by SDA if, and only if, it can be expressed as one of five kinds of thing. If it cannot, covering it would force cloud-specific knowledge into the engine, or make the tool guess — so it is out of scope.

1 · numeric

A quantity that combines along a path

e.g. add the latencies, take the smallest capacity, multiply the availabilities

2 · time

A question about behaviour over time

slow tails, queueing, warm-up, feedback — answered by simulation

3 · relational

4 · enumerate

A legality or compatibility fact

does this port fit that one; is this wiring allowed

A "which whole designs are valid" choice

generate every admissible architecture from a set of options

5 · optimize**A search for the best or smallest change**

the cheapest design that meets the targets; the minimal fix

Everything in the coverage table below maps to one of these five. This is what makes the boundary principled rather than a matter of taste: if a desired feature does not reduce to a quantity-with-a-rule, a time question, a legality fact, an enumeration, or an optimization, it does not belong in the engine.

4 What SDA covers today

Grouped by theme, in plain language, with the mechanism that makes each possible. Every item here is backed by an automated test or a worked real-world example.

Capacity and bottlenecks

SDA computes the throughput (requests per second) the design can sustain by taking the *smallest* capacity along each request path — the bottleneck — and then tracing which specific component is the binding constraint, so you know exactly where to add capacity. Where a component's capacity follows from its configuration (for example, the number of worker threads divided by how long each request takes — a relationship known as Little's law), the engine derives it rather than asking you to supply it. Hard limits such as a database's maximum connection count are modelled as plain data, so the classic "connection-pool exhaustion" failure shows up before you ship.

Latency, including the slow tail

Mean latency is computed by adding the per-component times along the synchronous path. But means hide the cases users actually feel, so SDA also runs a discrete-event simulation — a step-by-step model of requests arriving, waiting in queues, and being

served — to compute true percentiles: the 50th, 99th, and 99.9th percentile response times. This is the one lens a static diagram fundamentally cannot provide, and it routinely reveals a slow tail hiding behind a healthy-looking average.

Reliability — availability and durability

Availability (the fraction of time a component is up) and durability (the chance data survives) both compound by multiplication along the path, which is why a single un-replicated component drags the whole system down. SDA computes both and surfaces the weakest link, making single points of failure visible as a measurable drop rather than a vague worry.

Cost

Fixed monthly costs are summed across the design. Pay-per-use costs — where you are billed in proportion to actual traffic — are expressed as a formula of throughput, so the bill rises with load and falls to nearly nothing when idle, exactly as a serverless system behaves.

Structure and legality

Before any numbers are computed, the relational solver checks that the wiring is legal: that connected ports speak compatible protocols and point the right way. Placement rules — "every stateful service needs a replica", "these two components may not be combined" — are enforced as well, and a catalogue of protocols keeps the vocabulary consistent and catches typos.

Search — running the design in reverse

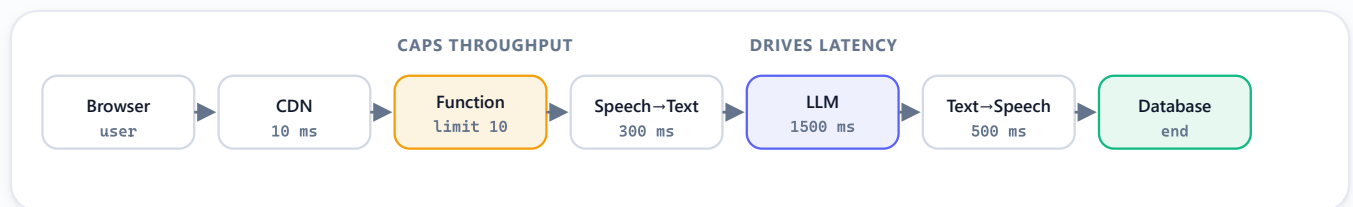
Beyond checking a design you drew, SDA can work backwards. **Optimize** finds the cheapest configuration that still meets every target. **Repair** finds the smallest change to a violating design that makes it pass. **Synthesize** generates whole candidate architectures from a set of component choices, evaluates each, discards those that miss the targets, and ranks the survivors. And when a target simply cannot be met, the engine explains which one and by how much, instead of returning a bare "impossible".

CAPABILITY	HOW IT IS COMPUTED	STATUS
Throughput & bottleneck	smallest capacity along the path, plus a trace to the binding component	● covered
Mean latency · slow-tail latency (p50/p99)	sum of times · discrete-event simulation	● covered
Availability · durability	product of per-component values along the path	● covered
Capacity from configuration (Little's law) · connection limits	a derived formula per component	● covered
Horizontal scaling (capacity grows with replicas)	a derived formula per component	● covered
Fixed cost · pay-per-use cost	sum · a formula of throughput	● covered
Verdict with a cause and a suggested fix	attribution over the computed model	● covered
Optimize · repair · explain an impossible target	constraint optimization	● covered
Synthesize & rank candidate architectures	enumeration, then evaluation, then ranking	● covered
Legal wiring (protocols, direction) · placement rules	relational checks · enumeration constraints	● covered
Synchronous vs asynchronous coupling · cold starts	path rules · simulation	● covered

5 A worked example

A real design, reconstructed from a live cloud project: a voice-conversation service in which a single "turn" — one spoken exchange — flows through six components.

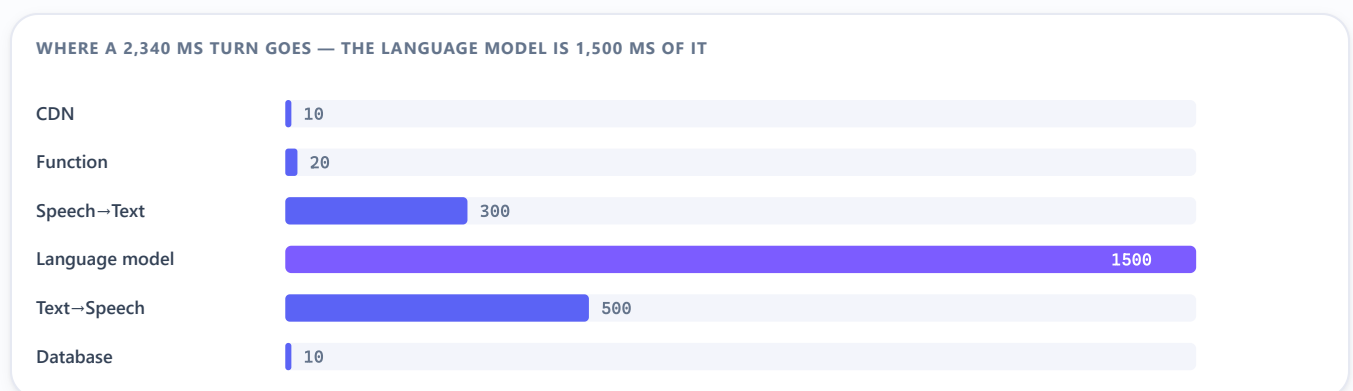
In this system a user speaks into a browser; the audio travels through a content-delivery network to a small serverless function, which orchestrates one turn of conversation: it sends the speech to a speech-to-text service, passes the text to a large language model that generates a reply, sends the reply to a text-to-speech service, and records the session in a database. The two questions that matter are: *how many conversations can run at once*, and *how long does a turn take*. The engine answers both, and the answers are not where intuition would put them.



The amber component caps how many conversations run at once; the indigo one dominates how long a turn takes.

Throughput. The serverless function is deliberately limited to ten simultaneous executions to cap cost. Because each turn keeps a function busy for the full ~2.3 seconds it takes the downstream services to respond, ten slots divided by 2.3 seconds is only about 4.3 turns per second — far below the eight the team needs for eight agents. The engine names the function as the binding constraint and suggests raising its concurrency limit.

Latency. The total time for a turn is 2,340 milliseconds, of which the language model alone is 1,500. The chart makes the point visceral: optimising the infrastructure (the network, the function, the database — all tiny bars) would barely move the total. The real lever is the model choice. The engine names the language model as the cause and points the fix there.



Each bar is one component's contribution to the turn, in milliseconds. The fix is the model, not the infrastructure.

Throughput: 4.3 turns per second, below the 8 required — ● **fails** the target; cause: the serverless function's concurrency limit; fix: raise it. **Latency:** 2,340 ms; cause: the language model; fix: choose a faster model.

6 The boundary of scope

Three honest categories: things mostly built, things one clean extension away, and things deliberately left out.

Partly covered — the mechanism exists, more content is needed

The simulation can in principle model failure dynamics such as retry storms (where retries pile load onto a struggling service), circuit breakers, and start-up transients, but the content describing those feedback loops is not yet written. Single points of failure are already visible through the availability calculation, though there is not yet a dedicated detector that flags them by name.

- **partial** Retry storms · circuit breakers · transients — simulation handles them; the models are unwritten.
- **partial** Single points of failure — surfaced via availability; no dedicated flag yet.

Within reach — feasible, one clean extension away

The strongest differentiators are about *correctness of data flow* rather than raw numbers: whether a path preserves a chosen consistency model (for instance, whether a user always reads their own most recent write), and whether message ordering is preserved end-to-end. Both are legality questions and fit the relational solver; they need one new capability — propagating a categorical label (not just a number) along a request path. Diagram tools cannot do this at all.

- **next** Consistency model (strong / eventual / read-your-writes).
- **next** Delivery and ordering guarantees (per-partition ordering, first-in-first-out queues).

Out of scope — deliberately

Some things are left out on purpose, because covering them would either dissolve the tool's discipline ("the engine computes, it does not guess") or turn it into a different product.

- **out** Identity and access management, and deep security — a large domain in its own right, deferred by decision.
- **out** Correctness of business logic or the data model — not a quantitative envelope.
- **out** Operational practice, monitoring runbooks, and organisational factors.
- **out** Exact live cloud pricing — the cost *structure* is modelled, but precise figures are a maintenance burden and are only ever shown marked as an estimate.

7 The honesty contract

THE TOOL MUST NOT LIE

Every value the tool reports is one of three things: an **assumption you supplied**, a **value it derived** from other values, or **unknown** — never a guess dressed up as a fact. Numbers that come from content are sourced from documentation where possible and marked as estimates otherwise. A tail-latency target is reported as *unknown* until a simulation has actually produced the figure. And where two solvers can answer the same question, they are tested against each other so a disagreement is caught immediately. This is the difference between "I drew an architecture" and "I verified an architecture".

8 Glossary

Every term and abbreviation used above, in plain language.

System design

The activity of choosing the components of a software system and how they connect, so that the whole meets goals for speed, scale, reliability and cost.

Component building block

One element of a design — a load balancer, a function, a database, a cache. In SDA each is described by data (its limits and formulas), never code.

Throughput

How much work a system handles per unit time, usually requests per second.

Latency

How long a single request takes from start to finish, usually in milliseconds.

Percentile · tail latency p50, p99, p999

A way to describe the spread of latencies. The 99th percentile (p99) is the time under which 99% of requests complete — the "slow tail" that the average hides. p50 is the median; p999 is the 99.9th percentile.

Availability

The fraction of time a component or system is working, often written as a number of "nines" (99.9% is "three nines").

Durability

The probability that stored data is not lost — for object storage, famously around "eleven nines".

Bottleneck

The single component that limits the whole system's throughput; the slowest link in the chain.

Capacity

The maximum throughput a single component can sustain.

Concurrency

How many requests a component handles at the same time — for example, a function's simultaneous-execution limit or a database's connection limit.

Little's law

A basic result of queueing theory: the average number of items in a system equals the arrival rate times the average time each spends inside. In practice it lets capacity be derived from concurrency and service time.

Service Level Objective SLO

A target the design must meet — for example "99th-percentile latency under 200 ms" or "throughput at least 1000 requests per second".

Band

How a target is shaped in SDA: a floor/target/ceiling range, an exact point, or a set of percentile targets.

Verdict

The engine's answer for one target: pass, warning, failure, or unknown — together with the cause and a suggested fix.

Cause-chain

The trace from a failing target back to the component actually responsible for it.

Remediation

A concrete, ranked suggestion for fixing a failed target (for example, "raise this component's concurrency").

Backpressure

What happens when a downstream component cannot keep up and the slowdown propagates upstream; it makes the maths circular and requires solving for a steady state.

Steady state · least-fixpoint

The stable operating point a system settles into once backpressure has propagated. "Least-fixpoint" is the mathematical name for the natural, smallest such point.

Discrete-event simulation DES

A step-by-step model of a system over time — requests arriving, waiting in queues, being served — used to compute true tail latencies and other time-dependent behaviour that

formulas cannot.

Queueing theory · M/M/1

The mathematics of waiting lines. M/M/1 is the simplest model (random arrivals, random service, one server) with known formulas, used to check that the simulation is correct.

Cold start

The extra delay when a component (often a serverless function) has to start up before serving a request; it affects the slow tail more than the average.

Synchronous vs asynchronous

A synchronous call makes the caller wait for the reply (its time adds to the total); an asynchronous one hands work off and returns immediately (it does not add to the caller's latency).

Protocol · port

A protocol is the language two components speak (HTTP, a database wire protocol). A port is a connection point on a component that speaks one protocol in one direction.

Manifest · registry · content

A manifest is the data file describing one component. The registry lists the properties the tool understands and how each combines along a path. Together they are the "content" that gives the engine its cloud meaning.

Domain-agnostic

Describes the engine: it computes over an abstract graph and contains no knowledge of cloud services — all such knowledge lives in content.

Aggregation algebra

The rule for how a property combines along a path: latency and cost add up, throughput takes the minimum, availability and durability multiply.

Optimize · repair · synthesize

The three "reverse" modes: optimize finds the cheapest design meeting the targets; repair finds the smallest change that fixes a failing one; synthesize generates and ranks whole candidate designs.

UNSAT

Short for "unsatisfiable": no configuration can meet the targets. SDA never stops there — it explains which target is impossible and by how much.

Solver

A reusable computation engine for one class of problem. SDA uses four: a numeric optimizer (MiniZinc), a simulator (discrete-event), a relational query engine (DataScript), and an enumerator (clingo / Answer Set Programming).